

Margin Allocation Desk — Technical Documentation

2026-09-21 · @Geet

A self-contained, client-side web app that models short-vol options margin usage across strategies and solves for the dollar-optimal allocation using a linear program, run entirely in the browser.

Architecture & runtime

Single HTML file (`index.html` , ~2,300 lines), no build step, no backend. All logic runs client-side in vanilla JS — no framework.

Concern	Approach
MILP solver	<code>javascript-lp-solver@1.0.3</code> , loaded from jsDelivr, branch-and-bound over a continuous LP
Excel read/write	<code>xlsx@0.18.5</code> (SheetJS), loaded from jsDelivr, for upload parsing and the sample-template download
Persistence	<code>localStorage</code> , scoped per-origin (<code>file://</code> vs <code>http://localhost</code> are separate buckets)
Styling	Hand-written CSS with light/dark theme via CSS custom properties, no framework

No network calls are made other than the two CDN library loads at page load. All strategy data, capital settings, and toggles stay in the browser — nothing is sent to a server. This makes the file fully portable: it can be opened directly (`file://`), served locally (`python3 -m http.server`), or shared as a zip, with identical behavior.

The whole app lives in one `<script>` block, organized into: state management (`loadState` / `saveState`), the margin model (`unitMargin` and its helpers), table rendering (`render` , `renderTotals`), the MILP builder (`buildAndSolve`), and UI wiring (event listeners at the bottom).

Data model

One `state` object holds everything and is persisted whole to `localStorage` (key `marginDeskStateV5`) on every change.

Global fields (`DEFAULTS` , each mirrored to a DOM input and re-synced on load so the browser's own form-memory can't silently override them):

Field	Meaning
<code>horizon</code>	<code>'weekly'</code> <code>'monthly'</code>
<code>marginInputMode</code>	<code>'dte'</code> <code>'independent'</code> — how per-weekday margin is derived
<code>objective</code>	<code>'total'</code> <code>'maximin'</code> <code>'riskadj'</code>
<code>totalCapital</code> , <code>minTotalMargin</code>	daily capital ceiling / floor, whole book
<code>nfExpiry</code> , <code>bnExpiry</code> , <code>snExpiry</code>	expiry weekday per index (<code>'mon'</code> - <code>'fri'</code>)
<code>monthlyExpiryWeek</code>	which of the 4 weeks (1-4) is the monthly-expiry week
<code>capNF/BN/SN</code> , <code>minNF/BN/SN</code>	per-index margin ceiling / floor
<code>riskLambda</code>	risk-aversion coefficient for the risk-adjusted objective
<code>offsetC</code> , <code>exponent</code>	DTE-curve calibration, shared by every strategy in <code>'dte'</code> mode

Per-strategy fields (`state.strategies[]`):

Field	Meaning
<code>name</code> , <code>index</code> (<code>NF</code> / <code>BN</code> / <code>SN</code>)	label and index
<code>overnight</code> (bool), <code>type</code> (<code>daily</code> / <code>weekly</code> / <code>monthly</code>)	holding behavior; <code>overnight:false</code> forces <code>type:'daily'</code>
<code>currentMargin</code>	today's actual \$ deployment (display only — not read by the solver)
<code>minMarginPerStrategy</code> , <code>maxMarginPerStrategy</code>	this strategy's own \$ floor/ceiling
<code>lotString</code>	5-digit string, lots per weekday at scale 1, e.g. <code>"12111"</code> (used when <code>marginInputMode === 'dte'</code>)
<code>margin0DTE</code>	\$ margin for one lot at zero days to expiry (<code>'dte'</code> mode)

Field	Meaning
<code>indepMargins</code>	<code>{mon,tue,wed,thu,fri}</code> object of independent \$ figures ('independent' mode)
<code>expectedReturn</code> , <code>risk</code>	\$ per one full run of the strategy (matching the horizon period), used only by the risk-adjusted objective

Nothing is deleted when switching `marginInputMode` or toggling objectives — both `lotString` / `margin0DTE` and `indepMargins` persist on every strategy regardless of which the model is currently reading, so switching back and forth never loses data.

Margin calculation

`unitMargin(strategy, slot)` returns the \$ margin for one unit of scale (see MILP formulation) on a given day-slot, and is the single function every other calculation reads from. It branches on `marginInputMode`:

DTE-derived mode — margin is worked out from a calibrated curve rather than entered directly per day:

$$\text{margin}(\text{DTE}) = \text{margin}_{\{0\text{DTE}\}} \times \left(\frac{\text{DTE} + c}{c} \right)^{\text{exponent}}$$

This is the same shape as the Brenner–Subramanyam approximation for an ATM straddle's price ($\approx 0.8 \times S \times \sigma \times \sqrt{T}$): expected move, and so margin, scales roughly with $\sqrt{\text{time}}$. `c` (default 1) keeps 0-DTE margin from collapsing to zero, since a real position always carries some floor margin even on expiry day; `exponent` (default 0.5) is the power-law fit. Both are global, calibrated once in the DTE Scaling tab against 2–3 real observed margin points, and shared by every strategy — this is a curve fit for interpolation within the calibrated range, not a physical law, and shouldn't be trusted to extrapolate.

`DTE` for a given weekday is computed against that strategy's index's expiry weekday, wrapping to the following week if the weekday falls after this week's expiry:

$$\text{DTE}(\text{weekday}) = (\text{expiryIndex} - \text{weekdayIndex} + 5) \bmod 5$$

e.g. for a Tuesday-expiry index: Mon = 1 DTE, Tue = 0, Wed = 4, Thu = 3, Fri = 2 (Wed's position is priced against *next* Tuesday's expiry).

The raw per-weekday margin is then `margin(DTE) \times lots(weekday)`, where `lots(weekday)` comes from the strategy's 5-digit `lotString`.

Independent mode — skips the curve entirely: `raw = indepMargins[weekday]`, a plain \$ figure with no formula tying days together. A weekday with `0` is treated as no position that day.

Expiry-day doubling (applied identically in both modes, after the raw value above): if the weekday is that index's expiry day, the raw margin is doubled — India-specific exchange behavior where expiry-day margin runs $\sim 2\times$ normal. For `type: 'daily' / 'weekly'` strategies this doubling fires every week; for `type: 'monthly'` it only fires in the one week marked as the monthly expiry week (`monthlyExpiryWeek`), since a monthly position's expiry-day spike only happens once a month, not weekly.

Current NSE/BSE schedule used as defaults: NIFTY (weekly & monthly) → Tuesday, BANK NIFTY (monthly-only since the exchange's 2025 consolidation) → Tuesday, SENSEX (weekly & monthly) → Thursday — editable per the user's actual segment.

Horizon & day-slots

`getSlots()` generates the set of day-slots the model optimizes over:

- **Weekly:** `['mon', 'tue', 'wed', 'thu', 'fri']` — 5 slots, one representative week.
- **Monthly:** `w1_mon ... w4_fri` — 20 slots, 4 weeks $\times$ 5 weekdays.

If any strategy has `type: 'monthly'`, Weekly is disabled and the app auto-switches to Monthly — a monthly position genuinely spans multiple weeks, so a single-week optimization can't represent it.

Why Monthly collapses to two rows, not twenty. Under Monthly horizon, `daily / weekly` strategies double on their expiry weekday every week regardless of which week it is, while `monthly` strategies only double in the one designated `monthlyExpiryWeek`. The practical consequence: every non-expiry week is mathematically identical to every other non-expiry week (same strategies, same doubling rules, same result) — so the whole month reduces to exactly two distinct week-shapes: **the expiry week** and **one representative non-expiry week**. The "margin by strategy and day" report and the problem-formulation panel both exploit this to show 2 rows instead of 4, with a toggle to switch which one is displayed, rather than cramming 20 near-duplicate rows on screen.

MILP formulation

Built in `buildAndSolve()` and solved with `javascript-lp-solver` (branch-and-bound over an LP relaxation; in practice a pure LP here since no variable is constrained to be an integer — see note below).

Decision variables. One per strategy:

$$x_i \in \mathbb{R}_{\geq 0}$$

x_i is a *dollar-scale factor*: $x_i = 1$ means "run strategy i's configured lot pattern exactly as entered." It is deliberately continuous, not integer — an earlier integer-lots version was replaced because forcing whole-lot granularity added a layer of abstraction ("units") without matching how the person actually wants to think about sizing; the tradeoff is that a solved answer like $x_i = 1.18$ doesn't map onto a clean tradeable lot count.

Objective (one of three, chosen by the `objective` field):

Mode	Objective	Meaning
<code>total</code>	$\max \sum_i c_i \cdot x_i, c_i = \sum_d \text{unitMargin}(i,d)$	maximize total margin used across all day-slots
<code>maximin</code>	$\max z, \text{s.t. } z \leq \frac{\sum_i \text{unitMargin}(i,d) \cdot x_i}{\text{capital}} \quad \forall d$	maximize the worst day's utilization — spreads load evenly rather than piling into a few days
<code>riskadj</code>	$\max \sum_i (\text{return}_i - \lambda \cdot \text{risk}_i) \cdot x_i$	maximize risk-adjusted P&L; $\text{return}_i / \text{risk}_i$ must be \$ amounts for one full run of strategy i over one horizon period (a week, or the whole month), not percentages — see below

Constraints (all always active, regardless of objective):

$$\lfloor_d \leq \sum_i \text{unitMargin}(i,d) \cdot x_i \leq \text{capital} \quad \forall d$$

Daily margin used across every strategy, bounded above by `totalCapital` and, if set, below by `minTotalMargin`.

$$\text{minMargin}_i \leq \text{unitMargin}(i,d) \cdot x_i \leq \text{maxMargin}_i \quad \forall d \quad \text{where } \text{unitMargin}(i,d) > 0$$

Per-strategy floor/ceiling. The floor is only enforced on days the strategy actually has a nonzero lot/margin entry — a strategy with a 0 in its lot pattern on Tuesday isn't forced to conjure a position that day just because a floor was set.

$$\text{minMargin}_I \leq \sum_{i \in I} \text{unitMargin}(i,d) \cdot x_i \leq \text{maxMargin}_I \quad \forall d, \quad \forall I \in \{\text{NF}, \text{BN}, \text{SN}\}$$

Per-index floor/ceiling, summed across every strategy sharing that index.

All floors/ceilings are optional (0/blank = unconstrained). A floor set tighter than an overlapping ceiling makes the model infeasible — by design, since a floor is a hard requirement, not a preference. If a strategy has zero margin on every day (empty lot pattern or `margin@DTE = 0`) but a positive `riskadj` coefficient, nothing bounds `x_i` above and the solve is unbounded rather than infeasible; the fix is a per-strategy ceiling or fixing the strategy's margin inputs.

Solve time is effectively instant at this problem's scale (a handful of strategies × 5–20 slots → tens of variables, hundreds of constraints) — single-digit milliseconds in-browser, well under what a person would notice.

UI reference

Two tabs: **Workbench** (everything above) and **DTE Scaling** (live calibration of `c / exponent`, previewed against a sample margin figure — the two fields there are preview-only and don't themselves feed the solver).

Every panel on the Workbench collapses via a caret in its header (state not persisted across reloads). Every numeric result is formatted through `fmt()` / `fmtNum()`, which respect a global **units toggle** (`$` vs `$M`, top-right) — this only changes display formatting, never the raw values stored in input fields.

Key panels, in order:

1. **Capital & calendar** — total capital, per-index expiry weekdays, monthly-expiry-week selector.
2. **Optimization setup** — horizon toggle, objective toggle (with risk- λ input when risk-adjusted is selected), whole-book and per-index floor/ceiling fields, Solve button.
3. **Strategies** — the strategy table, with a toggle between DTE-derived and Independent-by-weekday input, plus "load sample strategies," "reset strategies" (armed via a click-then-confirm pattern on the button itself, not a native `confirm()` dialog — those are unreliable inside a sandboxed preview and can silently no-op), file upload, and "download sample.xlsx."
4. **Margin by strategy and day** — the main report: one row per strategy plus TOTAL/UTILIZATION, by weekday. A dropdown switches between *Current allocation* (from `currentMargin`) and *Optimal allocation* (from the last solve — shows a prompt to solve first if none exists yet). Under Monthly horizon, a second toggle switches between the expiry week and a representative non-expiry week.
5. **Problem formulation** — the actual LP instance, generated live from current inputs (not a static template) — real coefficients, real constraint list, updates on every edit.

6. **Solve summary** — objective value, average utilization (current → optimal), solve time, populated after a solve and re-rendered (without re-solving) whenever the units toggle changes.

Toasts (bottom of screen) replace all native `alert()` / `confirm()` calls, for the same sandboxed-iframe reliability reason noted above. Info tooltips (? icons) are positioned with `position: fixed` computed from the icon's bounding rect, rather than CSS-relative to an ancestor — several panels scroll horizontally (`overflow-x: auto`), which per the CSS spec forces `overflow-y` to also clip, silently hiding any tooltip positioned relative to that ancestor; fixed positioning sidesteps this entirely.

File I/O

Upload accepts `.xlsx` , `.xls` , or `.csv` , parsed with SheetJS. Column headers are matched case/space-insensitively (`normalizeHeader()` lowercases and strips whitespace/underscores), so `Current Margin` , `current_margin` , and `currentmargin` all resolve the same way. Rows whose `index` isn't `NF` / `BN` / `SN` are silently dropped.

Column	Maps to	Notes
<code>strategy</code>	<code>name</code>	
<code>index</code>	<code>index</code>	must be NF/BN/SN
<code>overnight</code>	<code>overnight</code>	<code>y / yes / true / 1</code> → true
<code>type</code>	<code>type</code>	forced to <code>daily</code> if <code>overnight</code> is false
<code>current_margin</code>	<code>currentMargin</code>	
<code>min_margin</code> , <code>max_margin</code>	<code>minMarginPerStrategy</code> , <code>maxMarginPerStrategy</code>	
<code>lots</code>	<code>lotString</code>	digits only, truncated to 5
<code>margin_0dte</code>	<code>margin0DTE</code>	
<code>margin_mon ...</code> <code>margin_fri</code>	<code>indepMargins.{day}</code>	only used in Independent mode
<code>expected_return</code> , <code>risk</code>	<code>expectedReturn</code> , <code>risk</code>	only used by the risk-adjusted objective

"Download sample .xlsx" generates a workbook with exactly these headers via `XLSX.utils.json_to_sheet` + `XLSX.writeFile`, pre-filled with example rows — the reliable way to get a correctly-shaped starting file rather than building one by hand.